Introduction to NumPy Arrays and Matrices

Devi Prasad M

August 2, 2026

1 Working with ndarray

An *ndarray* object is the core data structure in NumPy, and it can represent vectors, matrices, and higher-dimensional arrays. NumPy arrays are *fixed-size*, homogeneous data structures.

1.1 Creating a 1D array

```
a = np.array(np.arange(1, 25, 1, np.int8))
assert(a.shape == (24,))
assert(a.size == 24)
assert(len(a) == 24)
assert(a.ndim == 1)
```

ndim represents the number of dimensions (axes) of the array. In this case, since a is a 1D array.

The *len* function returns the number of elements along the first axis of an array. For a 1D array, this is simply the total number of elements in the array.

1.2 Reshaping a 1D array into a 2D array

The *reshape* method is used to change the shape of an array without changing its data. The total number of elements in the reshaped array must match the total number of elements in the original array.

```
a = np.array(np.arange(1, 25, 1, np.int8))
assert (a == [x for x in range(1, 25)]).all()
b = a.reshape(2, 12)
assert(b.shape == (2, 12))
assert(b.size == 24)
assert(len(b) == 2)
assert(b.ndim == 2)
```

In the following we create a 2D array with 4 rows and 6 columns. The total number of elements in the reshaped array must match the total number of elements in the original 1D array.

```
a = np.array(np.arange(1, 25, 1, np.int8))
assert (a == [x for x in range(1, 25)]).all()
c = a.reshape(4, 6)
assert(c.shape == (4, 6))
assert(c.size == 24)
assert(len(c) == 4)
assert(c.ndim == 2)
```

1.3 Accessing rows and columns in a 2D array

```
# Check the first row of the reshaped array
assert (b[0] == [x for x in range(1, 13)]).all()
# Check the second row of the reshaped array
assert (b[1] == [x for x in range(13, 25)]).all()
# Check the first column of the reshaped array
assert (b[:, 0] == [1, 13]).all()
assert np.array_equal(b[:, 0], [1, 13])
# access the second column
assert np.array_equal(b[:, 1], [2, 14])
# the last column
assert np.array_equal(b[:, 11], [12, 24])
```

1.4 Reshaping the 2x12 array into a 2x3 array

We can further reshape the 2x12 array into a 2x3 array by slicing the first three columns of the 2x12 array.

```
a2x3 = b[:, 0:3]
assert(a2x3.shape == (2, 3))
assert(a2x3.size == 6)
assert(len(a2x3) == 2)
assert(a2x3.ndim == 2)
# access the last column of the 2x3 array
assert np.array_equal(a2x3[:, 2], [3, 15])
```

2 Combining NumPy arrays

Two *ndarray* objects can be combined using the *concatenate* function. This function takes a sequence of arrays and joins them along an existing axis. The arrays must have the same shape, except in the dimension corresponding to the axis along which they are concatenated.

```
a = np.concatenate(([1,2,3], [4,5], [7], [8,9], []), axis=0)
assert(np.array_equal(a, [1, 2, 3, 4, 5, 7, 8, 9]))
```

Efficiency Note

NumPy arrays have a fixed size: once an array is created, its shape cannot be changed in place. Therefore, operations such as repeated concatenation are inefficient - each concatenation creates a new array and copies the accumulated data. When building an array incrementally, it is more efficient to first collect the data in a Python list and create a NumPy array from that list.